

Semesterprüfung HS23/24
Studienstandort Rapperswil

NAME: _____ **VORNAME:** _____

MATRIKEL-Nr.: _____

STUDIENGANG: _____

Modulcode:	M_WE1
Modulbezeichnung:	WEB ENGINEERING 1
Modulowner:	INFORMATIK
Modulverantwortung:	Frieder Loch
Prüfungsverantwortliche/r:	Frieder Loch
Datum/Zeit:	Donnerstag, 25.01.2024, 08:10 bis 10:10 Uhr

erreichte / max. Punktzahl: _____

(Beispiel: erreichte / max. Punktzahl: 45 / 60)

Prüfungsnote: _____

Korrektur: _____

(Name/Unterschrift)

Prüfungsinformationen

MIT LÖSUNG

Zeit: 120 Minuten

Erlaubtes Hilfsmittel: 3 Blätter A4 (ein- oder doppelseitig beschrieben)

1. Legen Sie Ihre Legitimationskarte auf das Pult.
2. Mobiltelefone und andere elektronische Geräte ausschalten.
3. Tragen Sie Ihren Namen auf diesem Deckblatt ein.
4. Kontrollieren Sie, ob Sie alle Aufgabenblätter erhalten haben.
5. Streichen Sie ungültige Lösungsteile deutlich durch. Mehrfachlösungen sind falsch.
6. Zusatzblätter erhalten Sie bei Bedarf vom Prüfer. Verwenden sie kein eigenes Papier.
7. Geben Sie am Prüfungsende diese Lösung zusammen mit eventuellen Zusatzblättern ab.
8. Vorzeitige Abgabe bis maximal 15 Minuten vor Ende der Prüfung.
9. Bleiben Sie am Prüfungsende sitzen, bis alle Prüfungen eingesammelt wurden.
10. Der Weissraum lässt keine Rückschlüsse auf die Länge der erwarteten Antwort zu.
11. Beschriften Sie lose Blätter mit Ihrem Namen.
12. Schreiben Sie Ihre Lösung möglichst nicht mit Bleistift.
13. Verwenden Sie keinen roten Kugelschreiber.
14. Treffen Sie, falls notwendig, plausible Annahmen und vermerken Sie diese konkret.
15. Diese Prüfung hat einen Anhang mit nützlichen Informationen. Dieser kann abgetrennt werden.
16. Verstösse gegen Cleancode-Richtlinien können zum Punktabzug führen.

Eine erfolgreiche Prüfung wünschen
Frieder Loch und Michael Gfeller

#	Aufgabe	Punkte	Erreicht
1	HTML	13	
2	CSS	12	
3	Usability und Barrierefreiheit	10	
4	JavaScript	19	
5	Promises	12	
6	Anwendungsentwicklung	32	
	Total	98	

1 HTML (13P)

- a) Bei der Programmierung einer Webanwendung trennen wir **Inhalt**, **Darstellung** und **Verhalten**. Mit welcher grundlegenden Webtechnologie (HTML, CSS oder Javascript) werden diese Teile umgesetzt? Beschreibe am Beispiel einer ungeordneten Liste, welche Aspekte durch Inhalt, Darstellung und Verhalten beeinflusst werden. Schreibe **keinen** Code. (6P)

Inhalt

Technologie: ☐ HTML ☐ CSS ☐ Javascript

HTML. Der Inhalt beschreibt, dass ein Teil des Dokuments eine Liste ist und aus welchen Teilen die Liste besteht. (1P Technologie, 1P Beispiel)

Darstellung

Technologie: ☐ HTML ☐ CSS ☐ Javascript

CSS. Es wird beschrieben, wie die Liste und ihre Komponenten dargestellt werden sollen (z.B. Farbe).

Verhalten

Technologie: ☐ HTML ☐ CSS ☐ Javascript

Javascript. Es wird beschrieben, welche Aktionen bei einem Klick auf Elemente durchgeführt werden.

- b) Beschreibe den Unterschied zwischen **Syntax** und **Semantik** am Beispiel des <thead>-Elements. Unterscheide zunächst, was Syntax und Semantik in einem HTML-Dokument beschreiben. Analysiere die Spezifikation des Elements (siehe Abbildung 1) und beziehe dich bei deiner Lösung auf diese Spezifikation. Gib an, welche Elemente der Spezifikation des <thead>-Elements Syntax und Semantik beschreiben. Nenne jeweils **ein** Element als Beispiel für Syntax und Semantik. Es ist keine Begründung erforderlich. (4P)

--- Nutzen Sie für die Lösung die nächste Seite ---

<thead>: The Table Head element

The `<thead>` [HTML](#) element defines a set of rows defining the head of the columns of the table.

Content categories	None.
Permitted content	Zero or more <code><tr></code> elements.
Tag omission	The start tag is mandatory. The end tag may be omitted if the <code><thead></code> element is immediately followed by a <code><tbody></code> or <code><tfoot></code> element.
Permitted parents	A <code><table></code> element. The <code><thead></code> must appear after any <code><caption></code> or <code><colgroup></code> element, even implicitly defined, but before any <code><tbody></code> , <code><tfoot></code> and <code><tr></code> element.
Implicit ARIA role	<code>rowgroup</code>
Permitted ARIA roles	Any
DOM interface	HTMLTableSectionElement

Abbildung 1: Spezifikation des `<thead>`-Elements.

Syntax beschreibt:

Syntax beschreibt, wie der Code geschrieben sein soll (Regeln und Struktur)

Beispiel:

In einem `<thead>` können nur kein, ein oder mehrere `<tr>` vorkommen

Semantik beschreibt:

Die Semantik beschreibt, was ein Element bedeutet.

Beispiel:

`<thead>` bedeutet, dass der Inhalt der Header der Spalte ist.

- c) Kreuze bei den folgenden Fragen ausschliesslich die **eine korrekte Antwort** an.
(3P, jew. 1P)

Sie erstellen eine Blog-Seite. Wo würden Sie das `<article>`-Element verwenden? (1P)

- ☐ Für das Navigationsmenü, das auf verschiedene Blogbeiträge verweist.
- ☐ Für den Kommentarbereich unter jedem Blogbeitrag.
- ☐ Für die einzelnen Blogbeiträge. **Für die einzelnen Blogbeiträge**
- ☐ Für die Kopfzeile der Website.

Sie erstellen eine Webseite für ein Restaurant. Ist das `<aside>`-Element für die Öffnungszeiten auf einer Seite, die das Menü des Restaurants beschreibt, semantisch korrekt? (1P)

- ☐ Wahr **Wahr**
- ☐ Falsch

Ist das `<ul>`-Element für die Zutatenliste eines Rezepts semantisch korrekt? (1P)

- ☐ Wahr
- ☐ Falsch **Beide ok**

2 CSS (12P)

a) Kreuze bei den folgenden Fragen ausschliesslich die **eine korrekte Antwort** an. (4P)

Welcher Selektor hat die **höchste** Spezifität? (2P)

- ☐ #main .article p **#main .article p**
- ☐ div.article p
- ☐ #main article p
- ☐ .article p

Welche Spezifität hat der Selektor **#sidebar .widget h3::after** (2P)

- ☐ 1, 1, 1, 2
- ☐ 0, 1, 1, 2 **0, 1, 1, 2**
- ☐ 0, 0, 2, 1
- ☐ 0, 2, 1, 0

b) Schreibe möglichst kompakten CSS-Code, mit dem du die folgenden Eigenschaften erzielst. Es ist **kein** Selektor erforderlich. (2P)

- Breite: 200px
- Höhe: 150px
- Box soll einen schwarzen Rand von 2px besitzen.
- Hintergrundfarbe: lightgray

```
width: 200px  
height: 150px  
border: 2px black solid  
background: lightgray
```

c) Gegeben ist der folgende HTML-Code

```
<body>
  <div>
    <p>Absatz</p>
    <span>Span</span>
  </div>
</body>
```

Schreibe den CSS-Code, um das Layout in Abbildung 2 zu erhalten. Abhängig von deinem Lösungsweg, müssen eventuell nicht alle Lücken ergänzt werden. (6P)



Abbildung 2: Layoutvorlage.

```
div {
  background-color: lightgray;
  display: flex
}

p {
  background-color: green;
}

span {
  background-color: aqua;
}

div {
  display: flex;
  background-color: lightgray;
}

p {
  align-self: center;
  margin-right: 5px;
  background-color: green;
}

span {
  align-self: center;
  background-color: aqua;
}

1 Punkt für Margin
5 Punkte für die richtige
Anordnung
```

3 Usability und Barrierefreiheit (10P)

a) Die WCAG nennen vier Eigenschaften barrierefreier Webseiten.

- Wahrnehmbarkeit
- Bedienbarkeit
- Verständlichkeit
- Robustheit

Wähle **2 der 4** Eigenschaften. Erläutere deren Bedeutung und gib ein konkretes Beispiel, das eine gute Umsetzung zeigt. Das Beispiel muss konkret sein und HTML, CSS oder Javascript umfassen. (4P, jew. 2P)

Kategorie _____

Wahrnehmbarkeit

Information muss für alle Menschen,
unabhängig von ihren Fähigkeiten,
wahrnehmbar sein.

Beispiel: Img mit alt-tag

Robustheit:

Kategorie _____

Beispiel: Standardkonformes HTML

- b) Erkläre das **Gesetz der Nähe** und das **Gesetz der Ähnlichkeit** am Screenshot in Abbildung 3. Nenne die konkreten Elemente und erläutere kurz, warum diese eine gelungene Umsetzung darstellen (6P, jew. 3P)

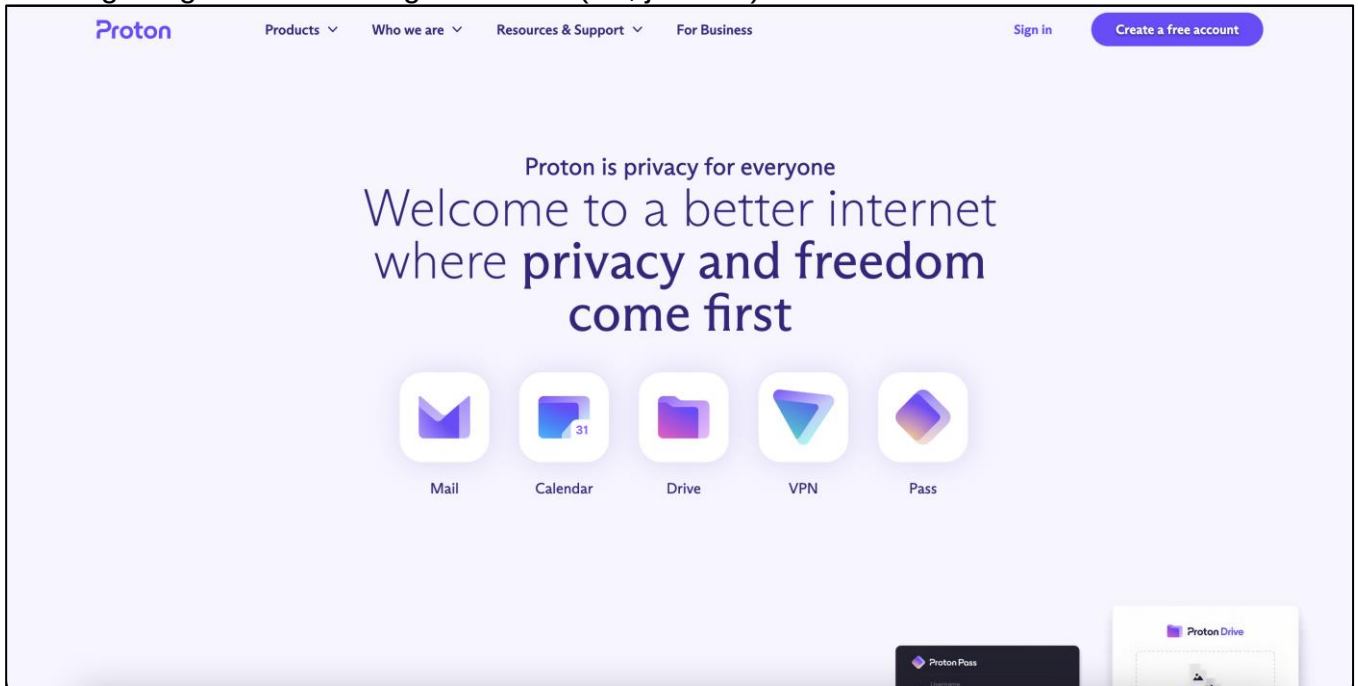


Abbildung 3: Screenshot einer Webseite.

Gesetz der Nähe

Ähnliche Dinge sind in der Nähe voneinander. Zum Beispiel Menüoptionen und die Links im Menü.

Gesetz der Ähnlichkeit

Die Symbole für die Dienste und Services sind optisch ähnlich gestaltet.

4 JavaScript (20P)

4.1 Theorie (9P)

a) Notieren Sie das Resultat bzw. die Ausgabe des jeweiligen Ausdrucks. (jew. 1P)

Ausdruck	Resultat
"5" * "a" - 5	„NaN“
false == '' == 0	False
[1, 2] + [3, 4]	1,23,4
const obj1 = {a: "1", b: "2"} const obj2 = {b: "3", c: "4"} console.log({...obj1, obj2});	{ a: '1', b: '2', obj2: { b: '3', c: '4' } }
[1,2,3].map((x, i) => i)	0,1,2 oder [0,1,2]

4.2 Module

Module in JS kennen zwei unterschiedliche Syntaxe um Werte zu exportieren. Wie lauten diese? (1P)

```
// Default Export und Named export
```

b) Welche Variante ist zu bevorzugen? Begründen Sie ihre Antwort (1P)

```
// Persönliche preferenz; sich an einen Stil halten
```

4.3 Array: Programmieren (10P)

Ergänzen Sie folgende `'calculate'`-Funktion damit diese korrekt funktioniert.

- Die Funktion soll von einer Liste von Objekten die jeweiligen Felder summieren bzw. den Durchschnitt bilden.
- Falls ein Wert `null` / `undefined` ist soll dieser ignoriert werden.
- Keine `if else` / `for` / `while` / `switch case`.
- Nutzen Sie passende Array-Methoden.
- Zusätzliche Funktionen / Objekte sind erlaubt.

Beispiel Daten:

```
const items = [  
  {name: "Ananas", price: 3, amount: 5}, {name: "Shirt", price: 30, amount: 1},  
  {name: "Auto", price: 50, amount: 1}, {name: "Bread", price: 1, amount: 10},  
  {name: "Cheese", price: null, amount: undefined}];
```

Beispiel Aufruf:

```
console.log(calculate(items, "sum", "price")); // 84  
console.log(calculate(items, "sum", "amount")); // 17  
console.log(calculate(items, "sum", "name")); // NaN  
console.log(calculate(items, "avg", "price")); // 21  
console.log(calculate(items, "avg", "amount")); // 4.25  
console.log(calculate(items, "avg", "name")); // NaN
```

```
const calculate = (items, methode, field ) => {
```

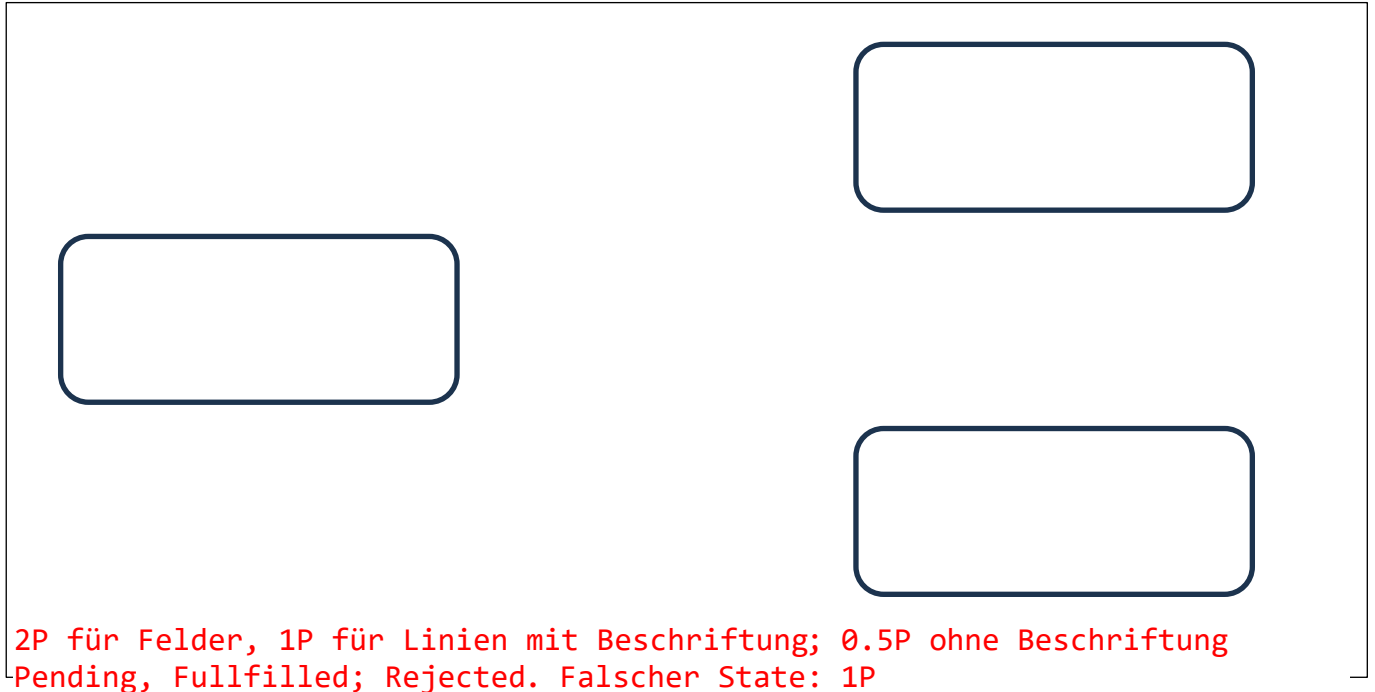
```
}
```

```
const actions = { 2P für action map:
  "sum": (array) => array.reduce((acc, cur) => acc + cur ), 2P für sum()
  "avg": (array) => { 1P für sum, falls kein copy & paste -> 0P
    return actions["sum"](array) / array.length;
  },
}

const calculate = (items, methode, field ) => {
  const values = items.map(x => x[field]).filter(x => typeof x === 'number' ); // 2P Value; filter ist
  // der wichtige Teil;
  if(values.length === 0){ // 2P check
    return NaN;
  }
  return actions[methode](values) // 1P Aufruf
}
```

5 Promises (12P)

a) Nenne die **drei Zustände**, in denen sich ein Promise befinden kann. Zeichne die Übergänge zwischen den Zuständen ein und bezeichne diese. (3P)



2P für Felder, 1P für Linien mit Beschriftung; 0.5P ohne Beschriftung
Pending, Fullfilled; Rejected. Falscher State: 1P

b) Was ist die Ausgabe?

Kreuze bei den folgenden Fragen die eine korrekte Antwort an.

```
const myPromise = Promise.resolve('Promise has resolved.');
```

```
setTimeout(() => {  
  console.log(myPromise);  
}, 3000);
```

- ☐ Promise has resolved.
- ☐ undefined
- ☐ Ein Fehler wird geworfen
- ☒ Promise { 'Promise has resolved.' }

c) Was ist die Ausgabe? (3P)

```
Promise.resolve(5)  
  .then(value => value * 2)  
  .then(value => console.log(value));
```

- ☐ 5
- ☒ 10
- ☐ undefined
- ☐ Eine Fehlermeldung

d) Was ist die Ausgabe? (3P)

```
Promise.reject('Error occurred')  
  .catch(err => console.log(err))  
  .then(() => console.log('Resolved'));
```

- ☒ **Error occurred dann Resolved**
 - ☐ **Resolved dann Error occurred**
 - ☐ **Nur Error occurred**
 - ☐ **Nur Resolved**
-

6 Anwendungsentwicklung (32P)

a) Die folgenden Aussagen beschreiben Eigenschaften von HTTP-Methoden. Kreuze **ausschliesslich die korrekten Aussagen** an. (4P)

- ☐ Sichere Methoden rufen Daten ab und haben keine Seiteneffekte auf dem Server.
- ☐ Idempotente Methoden können den Zustand des Servers ändern, aber mehrfache identische Anfragen haben den gleichen Effekt wie eine einzelne Anfrage.
- ☐ Die GET-Methode ist sicher und idempotent.
- ☐ Die POST-Methode ist sicher, da sie zum Übermitteln von Daten an einen Server zur Verarbeitung verwendet wird. **Falsch**
- ☐ GET, PUT und DELETE sind idempotent, wobei PUT und DELETE den Serverzustand ohne zusätzliche Effekte bei wiederholten Anfragen ändern können.
- ☐ Sichere Methoden können Seiteneffekte haben, wenn diese für den Client nicht sichtbar sind. **Falsch**

Pro falsches Kreuz: 1P weniger; mindestens 0 Punkte

b) In den folgenden Aufgaben werden Komponenten für ein Haushaltsbuch entwickelt. Das Haushaltsbuch beschränkt sich auf einen Saldo (Kontostand) von Einnahmen und Ausgaben. Zunächst wird das Backend entwickelt.

Entwickeln Sie die Funktion `handleIncome`. Die Methode nimmt Einnahmen entgegen und aktualisiert den Kontostand. (8P)

- Die Methode erhält das request- und response-Objekt und aktualisiert den Kontostand. Verwenden Sie für das Speichern des Kontostandes im Backend die bereits deklarierte Variable `balance`.
- Der Einnahmenbetrag wird als URL-Parameter (`value`) übergeben. Es sind nur Ganzzahlen erlaubt.
- Je nachdem, ob ein gültiger oder ein ungültiger Parameter übergeben wurde, werden entsprechende http-Statuscodes zurückgegeben.
- Mit der Response wird der neue Kontostand als Text an den Client zurückgegeben.

```
let balance =           // Deklaration

const handleIncomes = (req, res) => {
  const queryString = url.parse(req.url).query;
  const queryParams = new URLSearchParams(queryString);

  let amount = parseInt(queryParams.get('value'), 10);2P

  if (!isNaN(amount)) { 1P
    balance += amount; 1P ( inkl. Init Wert )
    res.writeHead(200, {'Content-Type': 'text/plain'}); 1P
    res.end(`${balance}`); 1P
  } else { 1P
    res.writeHead(400); 1P
    res.end('Invalid amount' ); 1P
  }

};
```


- c) Abbildung 4 zeigt das Frontend. Schreibe den HTML-Code, der die Struktur des Frontends abbildet. Es ist nicht erforderlich, ID- oder class-Attribute zu setzen. (8P)

Expense Tracker

User

Update User

Balance

Get Current Balance

Balance: \$0

Transaction

Add Expense

Add Income

Abbildung 4: Frontend des Haushaltsbuchs.

```
<main> 1.5P
  <h1>Expense Tracker</h1>
  <section>
    <h2>User</h2>
    <input type="text" placeholder="Enter your name"> 1P
    <button >Update User</button>
  </section>
  <section>
    <h2>Balance</h2>
    <button>Get Current Balance</button> 1P
    <p>Balance: $0</p> 0.5P
  </section>
  <section>
    <h2>Transaction</h2>
    <input type="number" placeholder="Enter amount"> 1P
    <button>Add Expense</button> 1P (für beide)
    <button>Add Income</button>
  </section>
</main>
```

1P: sections

1P: title

Für Input muss alles korrekt sein; damit es einen Punkt gibt;

- d) Implementiere die Methode `fetchRequest`. Die Methode ist wiederverwendbar und sendet einen http-Request an einen bestimmten Endpunkt. Die Methode erhält drei Parameter:
- **method**: http-Methode für den Request.
 - **url**: URL, an die der Request gesendet werden soll.
 - **data**: Daten, die mit dem Request gesendet werden sollen.

Die Methode gibt die Response als JSON zurück. (8P)

```
function fetchRequest(method, url, data = null) { 1P
  const options = { 2P
    method: method,
    headers: { 'Content-Type': 'application/json', 'mode': 'no-cors' },
  };
  if (data) { 1P
    options.body = JSON.stringify(data); 1P
  }
  return fetch(`http://localhost:8080/${url}`, options) 1P
    .then(response => response.json()); 2P
}
-1...-3...für falsche Code Zeilen
```

```
function fetchRequest(                                     ) {
```

- e) Schreibe nun die Methode `addTransaction`. Diese liest den Betrag aus dem Eingabefeld aus und sendet einen POST-Request an den Server. Der Wert aus dem Eingabefeld wird als URL-Parameter transferiert. Nach Abschluss des Requests wird der aktuelle Betrag in das entsprechende Feld im Frontend geschrieben. Zum Referenzieren der Felder können plausible Platzhalter als ID verwendet werden. (4P)

```
function addTransaction(type) {  
    const amount = document.getElementById('amount').value; 1P  
  
    const urlWithParam = type + '?amount=' + amount; // URL: 1P  
  
    fetchRequest('POST', urlWithParam) 1P  
        .then(balanceData => {  
            document.getElementById('balance').innerText = 'Balance: $' +  
balanceData.balance; // 1P  
        })  
}  
-1...-3...für falsche Code Zeilen  
function addTransaction(                                     ) {
```

-----1-----

```
let amount = parseInt(queryParams.get('value'), 10);2P

if (!isNaN(amount)) { 1P
  balance += amount; 1P ( inkl. Init Wert )
  res.writeHead(200, {'Content-Type': 'text/plain'}); 2P
  res.end(`${balance}`); 1P
} else {
  res.writeHead(400); 1P (beide)
  res.end('Invalid amount' );
}
```

-----2-----

```
<main> 2P (mit h1, ohne main ->0P)
  <h1>Expense Tracker</h1>
  <section> 2P
    <h2>User</h2>
    <input type="text" placeholder="Enter your name"> 1P
    <button >Update User</button>
  </section>
  <section> 2P
    <h2>Balance</h2>
    <button>Get Current Balance</button>
    <p>Balance: $0</p>
  </section>
  <section> 2P
    <h2>Transaction</h2>
    <input type="number" placeholder="Enter amount"> 1P
    <button>Add Expense</button>
    <button>Add Income</button>
  </section>
</main>
```

Pro Fehler in der Section: 2 1 0.5 0P
Section -> article -> einmalig 1P weniger;

-1 für

-----3-----

```
function fetchRequest(method, url, data = null) { 1P
  const options = { 2P (1P, falls inline)
    method: method,
    headers: { 'Content-Type': 'application/json' },x
  };
  if (data) { 1P
    options.body = JSON.stringify(data); 1P
  }
  return fetch(`http://localhost:8080/${url}`, options) 1P
    .then(response => response.json()); 2P
}
```

-1...-3...für falsche Code Zeilen

-----4-----

```
function addTransaction(type) {
  const amount = document.getElementById('amount').value; 1P

  const urlWithParam = type + '?amount=' + amount; // URL: 1P

  fetchRequest('POST', urlWithParam) 1P (mit promise oder korrektem async)
    .then(balanceData => {
      document.getElementById('balance').innerText = 'Balance: $' +
balanceData.balance; // 1P
    })
}
```

-1...-3...für falsche Code Zeilen

- Sichere Methoden rufen Daten ab und haben keine Seiteneffekte auf dem Server. ¶
- Idempotente Methoden können den Zustand des Servers ändern, aber mehrfache identische Anfragen haben den gleichen Effekt wie eine einzelne Anfrage. ¶
- Die GET-Methode ist sicher und idempotent. ¶
- Die POST-Methode ist sicher, da sie zum Übermitteln von Daten an einen Server zur Verarbeitung verwendet wird. ¶ **Falsch ¶**
- GET, PUT und DELETE sind idempotent, wobei PUT und DELETE den Serverzustand ohne zusätzliche Effekte bei wiederholten Anfragen ändern können. ¶
- Sichere Methoden können Seiteneffekte haben, wenn diese für den Client nicht sichtbar sind. ¶ **Falsch ¶**

Pro falsches Kreuz: 1P weniger; mindestens 0 Punkte ¶

7 Anhang

7.1 ul

Content categories	Flow content , and if the <code></code> element's children include at least one <code></code> element, palpable content .
Permitted content	Zero or more <code></code> , <code><script></code> and <code><template></code> elements.
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts flow content .
Implicit ARIA role	list

7.2 li

Content categories	None.
Permitted content	Flow content .
Tag omission	The end tag can be omitted if the list item is immediately followed by another <code></code> element, or if there is no more content in its parent element.
Permitted parents	An <code></code> , <code></code> , or <code><menu></code> element. Though not a conforming usage, the obsolete <code><div></code> can also be a parent.
Implicit ARIA role	listitem when child of an <code>ol</code> , <code>ul</code> or <code>menu</code>

7.3 ol

Content categories	Flow content , and if the <code></code> element's children include at least one <code></code> element, palpable content .
Permitted content	Zero or more <code></code> , <code><script></code> and <code><template></code> elements.
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts flow content .
Implicit ARIA role	list

7.4 h1

Content categories	Flow content , heading content, palpable content.
Permitted content	Phrasing content .
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts flow content .
Implicit ARIA role	heading

7.5 img

Content categories	Flow content , phrasing content , embedded content , palpable content . If the element has a <code>usemap</code> attribute, it also is a part of the interactive content category.
Permitted content	None; it is a void element .
Tag omission	Must have a start tag and must not have an end tag.
Permitted parents	Any element that accepts embedded content.
Implicit ARIA role	- with non-empty <code>alt</code> attribute or no <code>alt</code> attribute: img - with empty <code>alt</code> attribute: presentation

7.6 span

Content categories	Flow content , phrasing content .
Permitted content	Phrasing content .
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts phrasing content , or any element that accepts flow content .
Implicit ARIA role	No corresponding role 

7.7 main

Content categories	Flow content , palpable content.
Permitted content	Flow content .
Tag omission	None; both the starting and ending tags are mandatory.
Permitted parents	Where flow content is expected, but only if it is a hierarchically correct main element  .
Implicit ARIA role	main